

CSC343H5 Lecture 3

Data Definition Language (DDL) continued + Data Manipulation Language (DML)

Naaz Sibia (naaz.sibia@utoronto.ca)

Admin Notes

Quercus is back!

... but I'm going to keep using that Github page until Github goes down.



Admin Notes 2: Anonymous Feedback

I really liked the short quiz questions we did at the start of lectures because it helps refresh the content from the previous lecture. One suggestion is to instead create the quiz questions on Kahoot and we can answer it from there instead of Piazza. It much more engaging and its also pretty easy to use.

Admin Notes 2: Anonymous Feedback

I really liked the short quiz questions we did at the start of lectures because it helps refresh the content from the previous lecture. One suggestion is to instead create the quiz questions on Kahoot and we can answer it from there instead of Piazza. It much more engaging and its also pretty easy to use.



QUIZ TIME!

When the math question is testing your English skills more than your math skills:



New Material!

DDL Continued: Foreign Keys

Last week, we spoke about how:

- Foreign keys are primary keys of another table
- Can be used to identify weak entities and relationships

```
CREATE TABLE Pokemon (  
    name VARCHAR(20) PRIMARY KEY,  
    species VARCHAR(20),  
    primary_type VARCHAR(20)  
);  
  
CREATE TABLE Teaches (  
    pokemon VARCHAR(20),  
    move VARCHAR(20),  
    level INTEGER,  
    PRIMARY KEY (pokemon, move),  
    FOREIGN KEY (pokemon) REFERENCES Pokemon (name)  
);
```

The pokemon name needs to exist in the Pokemon table!



We also spoke about ways to define foreign keys

```
CREATE TABLE Pokemon (  
    name VARCHAR(20) PRIMARY KEY,  
    species VARCHAR(20),  
    primary_type VARCHAR(20)  
);
```

```
CREATE TABLE Teaches (  
    pokemon VARCHAR(20),  
    move VARCHAR(20),  
    level INTEGER,  
    PRIMARY KEY (pokemon, move),  
    FOREIGN KEY (pokemon) REFERENCES Pokemon(name)  
);
```



This Week:

Enforcing Foreign-Key Constraints

If there is a foreign-key constraint from relation R to relation S, two violations are possible:

1. An insert or update to R **introduces values** not found in S.
2. A deletion or update to S causes some tuples of R to **“dangle.”**

An insert or update to R **introduces values** not found in S.

1) An insert or update to R introduces values not found in S.

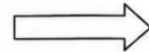
S (referenced table)

<u>s_id</u>	data
1	A
2	B
3	C

R (referencing table)

<u>r_id</u>	s_id (FK)	data
10	1	x
11	2	y

← R.s_id
references
S.s_id



INSERT or
UPDATE in R

<u>r_id</u>	s_id (FK)	data
10	1	x
11	2	y
12	99	z

99 not found
in S.s_id

→ FK violation

A deletion or update to S causes some tuples of R to “**dangle.**”

2) A deletion or update to S causes some tuples of R to “dangle.”

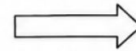
S (referenced table)

<u>s_id</u>	data
1	A
2	B
3	C

R (referencing table)

<u>r_id</u>	s_id (FK)	data
10	1	x
11	2	y
12	3	z

←
R.s_id
references
S.s_id



DELETE or
UPDATE in S

S (referenced table)

<u>s_id</u>	data
1	A
3	C

s_id = 2
deleted or
changed
(e.g., to 99)

R (referencing table)

<u>r_id</u>	s_id (FK)	data
10	1	x
11	2	y
12	3	z

no matching
row in S
→ dangling
tuple

Actions Taken

Example: suppose $R = \text{Borrows}$, $S = \text{Books}$

An insert or update to **Borrows** that introduces a nonexistent book must be rejected.

A deletion or update to **Books** that removes a book value found in some tuples of **Borrows** can be handled in three ways.

Actions Taken

DEFAULT: Reject the modification.



Why do more work when you can do less work?
- A wise person

Actions Taken

DEFAULT: Reject the modification.



Actions Taken

CASCADE: Do work... make the same changes in the **Borrows** table.

- **Deleted book:** delete the Borrows tuple with the book
- **Updated book:** change value in Borrows



DELETE in Books

BEFORE

Books (parent)

BookID	Title
1	1984
2	Brave New World
3	Dune

Borrows (child)

BorrowID	BookID (FK)	MemberID
101	1	10
102	2	11
103	3	12

FK

CASCADE

AFTER

Books

BookID	Title
1	1984
3	Dune

Borrows

BorrowID	BookID (FK)	MemberID
101	1	10
103	3	12

FK

✗

UPDATE in Books

BEFORE

Books (parent)

BookID	Title
1	1984
2	Brave New World
3	Dune

Borrows (child)

BorrowID	BookID (FK)	MemberID
101	1	10
102	2	11
103	3	12

FK

CASCADE

AFTER

Books

BookID	Title
1	1984
20	Brave New World
3	Dune

Borrows

BorrowID	BookID (FK)	MemberID
101	1	10
102	20	11
103	3	12

FK

- row deleted
- value changed

Reference Options

- **RESTRICT**: rejects the delete or update operation for the parent table.
- **CASCADE**: **ON DELETE** or **ON UPDATE** the row from the parent table, and automatically delete or update the matching rows in the child table.
- **SET NULL**: delete or update the row from the parent table, and set the Foreign Key column(s) in the child table to **NULL**.
- **NO ACTION**: equivalent to **RESTRICT**.

More details found in the [Postgres documentation](#).

Example: Setting Policy

```
CREATE TABLE Sells (  
    bar CHAR(20),  
    beer CHAR(20),  
    price REAL,  
    FOREIGN KEY (beer)  
        REFERENCES Beers (name)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE  
);
```



Two different choices!

Choosing a Policy

When we declare a foreign key, we may choose policies SET NULL or CASCADE independently for deletions and updates.

Follow the foreign-key declaration by:

ON [UPDATE, DELETE] [SET NULL CASCADE]

Two such clauses may be used.

Otherwise, the default (reject) is used.

Handout time!



Data Manipulation Language (DML)

SQL is a very-high-level language.

- Structured Query Language
- Say “what to do” rather than “how to do it.”
- Avoid a lot of data-manipulation details needed in procedural languages like C++ or Java.

DBMSs determine the “best” method of executing a query.
This is called “**query optimization**”.

Database Schemas in SQL

SQL is primarily a query language, used for retrieving data from a database.

- Data Manipulation Language (DML)

But SQL also includes a data-definition component for describing database schemas.

- Data Definition Language (DDL)

Select-From-Where Statements

- **SELECT** → desired attribute(s)
- **FROM** → one or more tables (i.e. entity-sets)
- **WHERE** → condition about tuples of the tables

Example

```
SELECT name, major  
FROM Students  
WHERE year = 2;
```

①

FROM

start with this table

Students

student_id	name	major	year
1	Ava	CS	1
2	Ben	Math	2
3	Chloe	CS	2
4	Diego	Bio	3

Example

```
SELECT name, major  
FROM Students  
WHERE year = 2;
```

② WHERE

keep rows that satisfy
the condition

Students (year = 2)

student_id	name	major	year
1	Ava	CS	1
2	Ben	Math	2
3	Chloe	CS	2
4	Diego	Bio	3

Example

```
SELECT name, major  
FROM Students  
WHERE year = 2;
```

③ SELECT
keep only the
needed columns

Result

name	major
Ben	Math
Chloe	CS

Overall Picture

```
SELECT name, major  
FROM Students  
WHERE year = 2;
```

① FROM

start with this table

Students

student_id	name	major	year
1	Ava	CS	1
2	Ben	Math	2
3	Chloe	CS	2
4	Diego	Bio	3

② WHERE

keep rows that satisfy
the condition

Students (year = 2)

student_id	name	major	year
1	Ava	CS	1
2	Ben	Math	2
3	Chloe	CS	2
4	Diego	Bio	3

③ SELECT

keep only the
needed columns

Result

name	major
Ben	Math
Chloe	CS

FROM
choose the table

WHERE
filter the rows

SELECT
choose the columns

RESULT
the final output

Our Schema

Our SQL queries will be based on the following database schema. Underlines indicate key attributes.

Students (sid, name, email)

Courses (cid, title, dept)

Instructors (name, office, phone)

Likes (student, course)

Teaches (instructor, course, semester)

Enrolled (student, course)

Using Courses(cid, title, dept)

```
SELECT title
FROM Courses
WHERE dept = 'Comp Sci';
```

<u>cid</u>	title	dept
CSC343	Databases	Comp Sci
CSC311	ML	Comp Sci
MAT223	Linear Algebra I	Math
CSC207	Software Design	Comp Sci
PSY 100	Intro Psychology	Psych
MAT235	Calculus II	Math

Using Courses(cid, title, dept)

```
SELECT title
FROM   Courses
WHERE  dept = 'Comp Sci';
```

<u>cid</u>	title	dept
CSC343	Databases	Comp Sci
CSC311	ML	Comp Sci
MAT223	Linear Algebra I	Math
CSC207	Software Design	Comp Sci
PSY 100	Intro Psychology	Psych
MAT235	Calculus II	Math

Note: These two are identical!

```
SELECT title  
FROM Courses  
WHERE dept = 'Comp Sci';
```

```
SELECT c.title  
FROM Courses c  
WHERE c.dept = 'Comp Sci';
```

Asterisks (*) in Select Clauses

When there is one relation in the FROM clause, * in the SELECT clause stands for "all attributes of this relation".

Example:

```
SELECT *  
FROM Courses  
WHERE dept = 'Comp Sci';
```

<u>cid</u>	title	dept
CSC343	Databases	Comp Sci
CSC311	ML	Comp Sci
MAT223	Linear Algebra I	Math
CSC207	Software Design	Comp Sci
PSY 100	Intro Psychology	Psych
MAT235	Calculus II	Math

Renaming Attributes

If you want the result to have different attribute names, use "AS <new name>" to rename an attribute.

```
SELECT cid AS course_code, title
FROM Courses
WHERE dept = 'Comp Sci';
```

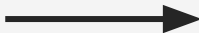
course_code	title
CSC343	Databases
CSC311	ML
CSC207	Software Design

You can also have expressions!

```
SELECT student, course, tuition*1.13 AS  
tuitionWithTax  
FROM Enrolled;
```

student	course	tuition
s001	CSC343	800
s002	CSC311	800
s003	MAT223	750
s004	CSC207	800

Query



student	course	tuitionWithTax
s001	CSC343	904.00
s002	CSC311	904.00
s003	MAT223	847.50
s004	CSC207	904.00

tuition in the table is **untouched**! the multiplication only happens in the result of the query.

And AS *tuitionWithTax* gives the computed column a readable name instead of something like ?column?.



Constants as Expressions

```
SELECT student, 'likes CSC343' AS whoLikesCSC343
FROM Likes
WHERE course = 'CSC343';
```

student	course
s001	CSC343
s001	CSC311
s002	CSC343
s003	MAT223
s004	CSC343

student	whoLikesCSC343
s001	likes CSC343
s002	likes CSC343
s004	likes CSC343

Constants as Expressions



The second column isn't pulled from the table at all! It's a constant string the query *invents* for every matching row.

```
SELECT student, 'likes CSC343' AS whoLikesCSC343
FROM Likes
WHERE course = 'CSC343';
```

student	course
s001	CSC343
s001	CSC311
s002	CSC343
s003	MAT223
s004	CSC343

student	whoLikesCSC343
s001	likes CSC343
s002	likes CSC343
s004	likes CSC343

That's it for now!

Next week: querying multiple tables and using logical operators!